

ATATÜRK ÜNİVERSİTESİ

Mühendislik Fakültesi
Bilgisayar Mühendisliği Bölümü
2025-2026 Güz Dönemi
MBM203/1275380/Veri Yapıları ve Algoritmalar Dersi

"Dogruluk" adlı veri yapısının sunumu

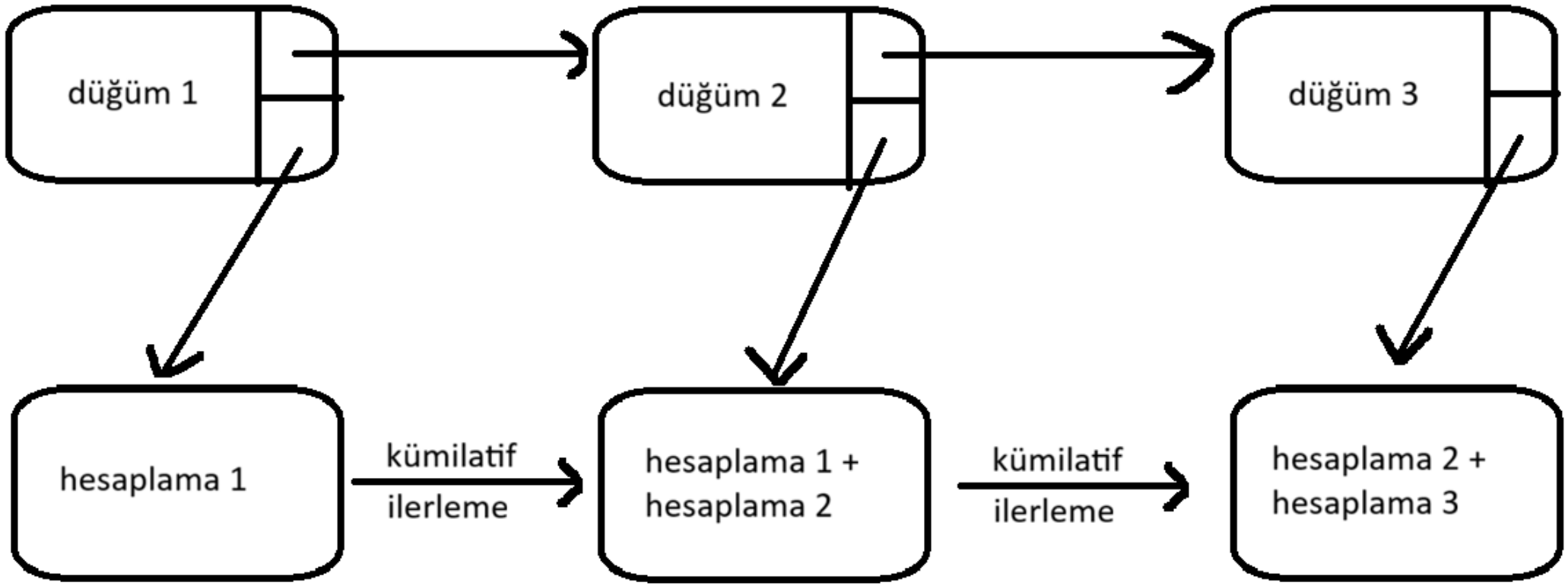
Öğrenci Bilgileri

Hüseyin Emir AKAY
240707032
Örgün Öğretim

Öğretim Üyesi Bilgileri

Dr. Öğr. Üyesi Esra ÇELİK

Veri yapısının çalışma mimarisi



Hesaplama düğüm içinde değil, yol üzerindeyken oluyor. Gerçek hayatta ki gibi.

Veri Yapısının Özellikleri

- Bu projede, duraklar arası akışı modellemek amacıyla tek taraflı bağıl listelerden oluşan bir kuyruk yapısı oluşturulmuştur. Düğüm yapısını tutmak için bağıl listelerden oluşma tek taraflı bir kuyruk yapısı tercih edilerek, durakların sıralı ve yönlü bir şekilde ilerlemesi sağlanmıştır. Her bir düğüm, yalnızca kendi durak kimliğini ve bir sonraki düğümün adresini tutmakta olup, zaman ve gecikmeye ilişkin herhangi bir hesaplama verisi doğrudan düğüm içinde saklamamaktadır.

- Oluşturulan yapı, zaman ve gecikme hesaplamalarını düğümler üzerinde değil, iki durak arasındaki geçişi temsil eden ayrı bir veri yapısı üzerinden gerçekleştiren özgün bir model sunmakta; her geçiş, oluşturulduğu anda bir önceki geçişin kümülatif sonucunu referans alarak kendi değerini üretmekte ve böylece sistem, yeni duraklar eklendikçe önceki durakların etkisini zincirleme biçimde hesaba katan bir zaman tahmin mekanizması oluşturmaktadır.

Diğer Veri Yapıları İle Karşılaştırımlası

Özellik	Dizi	Bağlı Liste	Yığın	Kuyruk	Ağaç	Graf	Dogruluk Yapısı
Sıralı Veri Tutma	Var	Var	Var (LIFO)	Var (FIFO)	Yok	Yok	Var
Dinamik Ağırlık/Katsayı	Yok	Yok	Yok	Yok	Kısmi	Statik	Var (Dinamik)
Kümülatif Hesaplama	Yok	Yok	Yok	Yok	Yok	Kısmi	Var (Otomatik)
Gerçek Zamanlı Adaptasyon	Yok	Yok	Yok	Yok	Kısmi	Kısmi	Var (Otomatik)

KOD KISMI

Tanımlamalar

"DurakDugumu" yapısı, sistemdeki her bir durağı temsil eder. Bu yapı yalnızca durağın kimlik bilgisini ve yapısal bağlantıları tutacak şekilde sade olarak tasarlanmıştır. Durak düğümü, zaman veya gecikmeye ilişkin herhangi bir hesaplanmış veri içermez. Böylece düğüm yapısı yalnızca konumsal bir temsil görevi üstlenir.

"DurakDugumu" içerisinde:
- Durağın numarasını temsil eden bir kimlik bilgisi
- Bir sonraki durak düğümünü gösteren pointer
- İlgili durak ile bir sonraki durak arasındaki geçişi temsil eden geçiş verisinin adresi bulunmaktadır.

Bu tasarım sayesinde hesaplama verileri düğümden ayrılarak geçiş bazlı bir model oluşturulmuştur.

```
                // DURAK DÜĞÜMÜ ( sadece konum )
struct DurakDugumu {
    int durakNO;              //   Durak numarasını tutar
    GecisVerisi* ileriGecis; //   hesaplama verisini işaret eder
    DurakDugumu* sonraki;    //  sonraki düğümü işaret eder

    //                     sade bir yapısı var
    //   çünkü Durak düğümü sadece kimlik ve adres bilgisi taşıyor.
    // Zaman, gecikme veya hesaplanmış herhangi bir değer burada değil
    //                   geçiş verisinin içinde.


    DurakDugumu(int no): durakNO(no), ileriGecis(nullptr), sonraki(nullptr) {}
};
```

"GecisVerisi" yapısı, iki durak arasındaki geçiş sırasında yapılan zaman ve gecikme hesaplamalarını temsil eder. Hesaplamalar düğümler üzerinde değil, bu yapı üzerinden gerçekleştirilir. Her geçiş verisi, oluşturulduğu anda kendi anlık süresini hesaplar ve bir önceki geçişin kümülatif sonucunu referans alarak yeni kümülatif süreyi üretir.

Bu yapı içerisinde:
- Anlık geçiş süresi
- Kullanıcıya gösterilen gelmesine kalan toplam süre
- Hesaplamada kullanılan saat bilgisi
- Yolcu yoğunluğunu temsil eden binemeyen yolcu sayısı saklanmaktadır.

"GecisVerisi" yapısı, önceki veya sonraki geçişleri doğrudan işaret eden bir pointer içermez ve durumdan bağımsız bir yapı sunar.

```
        // GEÇİŞ VERİSİ ( veri yolda hesaplanır )

struct GecisVerisi {
    double anlikSure;        // Hesaplama için kullanılır.
    double kumulatifSure;    // Gelmesine kalan toplam süre.
    int saat;                // saat verisini tutar.
    int binemeyen;           // binemeyen sayısını tutar.
};
```

"Dogruluk" sınıfı, sistemin ana kontrol yapısını oluşturur. Durak düğümlerinin eklenmesi, geçiş verilerinin oluşturulması, hesaplamaların yönetilmesi ve belleğin temizlenmesi gibi tüm işlemler bu sınıf üzerinden yürütülür. Sınıf, tek taraflı bağlı listelerden oluşan bir kuyruk yapısını temel alarak çalışır.

Bu sınıf içerisinde:
- Durak zincirinin başlangıç ve bitiş adresleri
- Bir önceki geçişin verisini tutan referans
- Sistem genelinde kullanılan saat ve yolcu yoğunluğu bilgileri saklanmaktadır.

Böylece hesaplamalar merkezi bir yönetici tarafından kontrol edilmektedir.

```
            // Ana veri yapısı
class Dogruluk {
private:

// Önemli olduklarından dolayı dışardan müdahaleye
//    kapatmak için "private" içinde tanımladım

 // sürekli bir döngü ile listenin sonuna gitmemek için
 DurakDugumu* bas;      // veri yapısının başlangıcı
 DurakDugumu* son;      // veri yapısının sonu
 GecisVerisi* sonGecis; // Önceki geçişi tutar

 int saat;
 int binemeyen;


       // Saat katsayısı
 double saatKatsayisi(int s) {
    if (s >= 7 && s <= 9)  return 1.3;
    if (s >= 12 && s <= 14) return 1.2;
    if (s >= 17 && s <= 19) return 1.4;
  return 1.0;}


       // Yolcu yoğunluğu katsayısı
  double kapasiteKatsayisi(int k) {
     if (k == 0) return 1.0;
     if (k <= 5) return 1.4;
     if (k <= 10) return 1.5;
    return 1.6;}

       // Geliş süresi hesaplama
 double anlikSureHesapla() {
    double sure = 1;
    return sure * saatKatsayisi(saat) * kapasiteKatsayisi(binemeyen) *2;}


public:
    Dogruluk(int s, int b): bas(nullptr), son(nullptr),
       sonGecis(nullptr),
       saat(s), binemeyen(b) {}
```

KOD KISMI

Fonksiyonlar

Bu fonksiyon, günün farklı saat dilimlerinde oluşabilecek yoğunluk etkisini modellemek amacıyla kullanılır. Girilen saat bilgisine bağlı olarak bir katsayı döndürerek, geçiş sürelerinin dinamik olarak ayarlanmasını sağlar.

```
      // Saat katsayısı
double saatKatsayisi(int s) {
    if (s >= 7 && s <= 9)  return 1.3;
    if (s >= 12 && s <= 14) return 1.2;
    if (s >= 17 && s <= 19) return 1.4;
 return 1.0;}
```

Durakta bekleyen yolcu yoğunluğunu temsil eden bu fonksiyon, binemeyen yolcu sayısına göre bir katsayı üretir. Yolcu yoğunluğu arttıkça gecikme süresinin artmasını düşünülerek kullanılmıştır.

```
    // Yolcu yoğunluğu katsayısı
double kapasiteKatsayisi(int k) {
    if (k == 0) return 1.0;
    if (k <= 5) return 1.4;
    if (k <= 10) return 1.5;
  return 1.6;}
```

Bu fonksiyon, iki durak arasındaki anlık geçiş süresini hesaplar. Hesaplama sırasında baz süre ile saat ve yolcu yoğunluğu katsayıları birlikte değerlendirilerek dinamik bir geçiş süresi elde edilir.

```
        // Geliş süresi hesaplama
double anlikSureHesapla() {
    double sure = 1;
    return sure * saatKatsayisi(saat) * kapasiteKatsayisi(binemeyen) *2;}
```

Bu fonksiyon, sisteme yeni bir durak düğümü ekler. Yeni durak eklendiğinde, bir önceki durak ile yeni durak arasındaki geçiş verisi oluşturulur. Bu geçiş verisi, önceki geçişin kümülatif sonucunu referans alarak kendi değerini üretir. Böylece sistem, duraklar ilerledikçe zincirleme biçimde güncellenen bir zaman tahmini üretir.

```
                    // DURAK EKLEME
void durakEkle(int no) {
    DurakDugumu* yeniDurak = new DurakDugumu(no);
    // yeni durak oluşturma

    if (bas == nullptr) { //Sona gelmişse
            bas = son = yeniDurak; }
    else {  // Sona gelmemişse
            GecisVerisi* gv = new GecisVerisi;

            gv->anlikSure = anlikSureHesapla();
            gv->saat = saat;
            gv->binemeyen = binemeyen;


            // birikimli hesaplama  ( gelmesine kalan dakika )

    if      (sonGecis == nullptr) // sona gelmişse
            gv->kumulatifSure = gv->anlikSure;
                 // kümilatif değeri son değere eşitler

    else      // Sona gelmemişse
            gv->kumulatifSure =
            sonGecis->kumulatifSure + gv->anlikSure;

            son->ileriGecis = gv;
            son->sonraki = yeniDurak;

            sonGecis = gv;
            son = yeniDurak;}
}
```

Bu fonksiyon, duraklar arasındaki geçişleri sıralı şekilde ekrana yazdırır. Kullanıcıya yalnızca “gelmesine kalan dakika” bilgisi sunularak, hesaplanan kümülatif zamanın anlaşılır bir biçimde gösterilmesi sağlanır.

```
        //YAZDIRMA
void yazdir() {
 DurakDugumu* temp = bas; // ilk değerden başlayarak

 cout << "\n--- DURAK BAZLI GELMESİNE KALAN SÜRE ---\n\n";

        // sonuna kadar ilerler
while (temp != nullptr && temp->sonraki != nullptr) {
    GecisVerisi* gv = temp->ileriGecis;

cout << "Durak " << temp->durakNO << " -> " <<
temp->sonraki->durakNO << " | Gelmesine Kalan Dakika: " <<
gv->kumulatifSure << " dk\n";

   temp = temp->sonraki;}
 }
```

Dinamik olarak oluşturulan tüm durak düğümleri ve geçiş verilerinin bellekten güvenli bir şekilde silinmesini sağlar. Program sonunda bellek yönetiminin doğru yapılması amacıyla kullanılmıştır.

```
        //BELLEK SİLME
void temizle() {
 while (bas != nullptr) {
 if (bas->ileriGecis != nullptr)
 delete bas->ileriGecis;

    DurakDugumu* temp = bas;
    bas = bas->sonraki;      // tüm yapıyı sırayla dolaşır
    delete temp;}

        son = nullptr;         // son kalan değerleride sıfıra eşitler
        sonGecis = nullptr;}
};
```

"main" fonksiyonu, kullanıcıdan gerekli girişleri alır, veri yapısını oluşturur, simülasyonu çalıştırır ve sonuçları ekrana yazdırarak programı sonlandırır.

```
    // ANA fonksiyon
int main() {
setlocale(LC_ALL, "Turkish");

int durakSayisi, saat, binemeyen;

cout << "\n\n\n";
cout << "Durak sayısı: ";
cin >> durakSayisi;
cout << "\n";

for (;;) {
    cout << "Saat (6-23): ";
    cin >> saat;
    cout << "\n";

    if (saat >= 6 && saat <= 23)
        break;   // geçerli değer alındığında döngüden çıkar
}

cout << "Durakta bekleyen yolcu sayısı: ";
cin >> binemeyen;
cout << "\n";

    Dogruluk sistem(saat, binemeyen);

    for (int i = 1; i <= durakSayisi; i++) {
        sistem.durakEkle(i);
    }
sistem.yazdir();     // değerleri yazdırır
sistem.temizle();    // belleğin içini boşaltır.

    return 0;}
```

Zaman Karmaşıklığı Analizi

İşlem (Fonksiyon)	Zaman Karmaşıklığı (Big O)	Açıklama
durakEkle(int durakNO)	O(1)	Son durak işaretçisi tutulduğu için tek adımda ekleme yapılır.
yazdir ()	O(N)	Tüm duraklar ve geçişler baştan sona gezilerek yazdırılır.
temizle ()	O(N)	Bellek temizliği için tüm düğümler tek tek silinir.
anlikSureHesapla ()	O(1)	Sabit sayıda matematiksel işlem ve koşul içerir.
saatKatsayisi ()	O(1)	Sabit sayıda karşılaştırma yapılır.
kapasiteKatsayisi ()	O(1)	Sabit sayıda koşul kontrolü yapılır.

Günlük Hayattan Örnek

Geliştirilen veri yapısı, bir toplu taşıma aracının duraklar arasında ilerlerken yaşadığı gecikmelerin tahmin edilmesine benzetilebilir. Yolcunun bekleme süresi yalnızca bir sonraki durağa olan mesafeye değil, önceki duraklarda oluşan yoğunlukların toplam etkisine de bağlıdır. Bu yapı, duraklar arasındaki gecikmeleri zincirleme biçimde birleştirerek kullanıcıya daha gerçekçi bir “gelmesine kalan dakika” bilgisi sunmayı kolaylaştırır.

Bilgisayar Bilimlerinden Örnek

Bir ağ paketinin toplam gecikmesi, geçtiği her bağlantıda oluşan gecikmelerin toplamına bağlıdır. Bu projede kullanılan veri yapısı da hesaplamayı düğümlerde değil, geçişler üzerinde gerçekleştirerek kümülatif gecikmenin zincirleme biçimde hesaplanmasını sağlar ve zaman tahmini gibi problemleri daha doğal bir şekilde modellemeyi kolaylaştırır ve daha doğru bir değer verir.